

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Научно-учебная лаборатория
методов анализа больших
данных
Стажер-исследователь

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / М.В. Минец /
«__» _____ 2026 г.

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.01-01 81 01–1 ЛУ

Исполнители:
студент группы БПИ238

_____ / М.М. Апаркин /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.01-01 81 01–1 ЛУ

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ**

Пояснительная записка

RU.17701729.09.01-01 81 01–1

Листов 24

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ «Пояснительная записка» разработан в соответствии с ГОСТ 19.404–79 и содержит обоснование принятых технических решений при разработке подсистемы управления мероприятиями, обработки платежей и генерации QR-кодов программного изделия «Система регистрации и управления жизненным циклом мероприятий».

Документ описывает назначение и область применения разработанных программных компонентов, алгоритмы функционирования микросервисов `events-service`, `payment-service` и `qr-service`, организацию входных и выходных данных, обоснование выбора технических и программных средств (PostgreSQL 16, Redis 7, Apache Kafka, MinIO, Prometheus, Grafana), а также ожидаемые технико-экономические показатели.

Документ составлен в соответствии с требованиями ГОСТ 19.105–78 и ГОСТ 19.404–79.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	2
2.1. Назначение программы	2
2.2. Область применения	2
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	3
3.1. Постановка задачи на разработку	3
3.1.1. Описание задачи	3
3.1.2. Описание применяемых методов	3
3.1.3. Ограничения и допущения	4
3.2. Описание алгоритма и функционирования программы	4
3.2.1. Архитектура events-service	4
3.2.2. Алгоритм обработки регистрации	5
3.2.3. Алгоритм работы ExpirationWorker	7
3.2.4. Настройка Apache Kafka	7
3.2.5. Настройка PostgreSQL	7
3.2.6. Настройка Redis	8
3.2.7. Интеграция с ЮKassa	8
3.2.8. Настройка MinIO	9
3.2.9. Настройка Prometheus и Grafana	10
3.3. Описание и обоснование выбора метода организации входных и выходных данных	11
3.3.1. Входные данные	11
3.3.2. Выходные данные	11
3.4. Описание и обоснование выбора состава технических и программных средств	12
3.4.1. Язык программирования: Go 1.24	12
3.4.2. СУБД: PostgreSQL 16	12
3.4.3. Кэш-хранилище: Redis 7	12
3.4.4. Брокер сообщений: Apache Kafka 7.4	13
3.4.5. Объектное хранилище: MinIO	13
3.4.6. Система мониторинга: Prometheus + Grafana	13
3.4.7. Сервер развёртывания	14
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	15
4.1. Показатели производительности	15
4.2. Показатели надёжности	15
4.3. Показатели ресурсоёмкости	16
4.4. Экономические показатели	16
5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	17

Ключевые проектные решения исполнителя	22
Контрольный перечень перед приемкой	23
Основания уникальности документа	23

1. ВВЕДЕНИЕ

Наименование программы: «Система регистрации и управления жизненным циклом мероприятий» (сокращённое обозначение — CSTATI).

Условное обозначение темы разработки по ГОСТ 19.103-77: **RU.17701729.09.01-01.**

Разработка ведётся на основании технического задания «Система регистрации и управления жизненным циклом мероприятий. Техническое задание» (обозначение: RU.17701729.09.01-01 ТЗ 01–1), утверждённого в 2025 г. на факультете компьютерных наук Национального исследовательского университета «Высшая школа экономики».

Исполнитель разработки описываемых компонентов — студент образовательной программы «Программная инженерия» факультета компьютерных наук НИУ ВШЭ, группа БПИ238, М.М. Апаркин.

Настоящая пояснительная записка охватывает следующие компоненты системы:

1. `events-service` — микросервис управления мероприятиями: API-сервер и Kafka-воркер обработки регистраций и платежей;
2. `payment-service` — микросервис обработки платежей с интеграцией платёжного шлюза ЮKassa;
3. `qr-service` — микросервис генерации QR-кодов электронных билетов.

Инфраструктурные компоненты, описываемые в данном документе: реляционная СУБД PostgreSQL 16, кэш-хранилище Redis 7, система потоковой передачи сообщений Apache Kafka 7.4, объектное хранилище MinIO, система мониторинга Prometheus совместно с Grafana.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

Комплекс программ «CSTATI» предназначен для автоматизации полного жизненного цикла публичного мероприятия: от анонса события и открытия онлайн-регистрации до подтверждения участия, приёма оплаты, выдачи электронного билета с персональным QR-кодом и контроля прохода на мероприятие.

Описываемые в настоящем документе компоненты выполняют следующие функции:

- `events-service` — центральный API-сервис: хранение информации о мероприятиях, волнах регистрации, категориях билетов, промокодах и регистрациях участников; координация рабочего процесса через Apache Kafka;
- `payment-service` — сервис, реализующий приём онлайн-оплаты через платёжный шлюз ЮKassa, обработку webhook-подтверждений и рассылку уведомлений;
- `qr-service` — сервис, генерирующий PNG-изображения QR-кодов для выданных билетов и сохраняющий их в объектном хранилище MinIO.

2.2. Область применения

Система предназначена для организации и проведения публичных мероприятий, требующих предварительной регистрации участников: конференций, митапов, хакатонов, воркшопов, образовательных интенсивов и корпоративных событий.

Типовые характеристики целевого мероприятия:

- количество участников: от 50 до 5 000 человек;
- наличие нескольких волн регистрации (ранняя регистрация, основная, VIP и т.д.);
- поддержка как бесплатных, так и платных билетов;
- необходимость контроля прохода на основании QR-кода.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку

3.1.1. Описание задачи

Требуется разработать подсистему управления жизненным циклом мероприятий в рамках микросервисной архитектуры. Подсистема должна:

1. предоставлять REST API для управления мероприятиями, волнами, билетами, промокодами и регистрациями;
2. обрабатывать события регистрации участников в асинхронном режиме через брокер сообщений;
3. интегрироваться с внешним платёжным шлюзом ЮKassa для приёма онлайн-оплаты;
4. генерировать QR-коды электронных билетов и сохранять их в объектном хранилище;
5. обеспечивать мониторинг состояния сервисов через систему сбора метрик.

3.1.2. Описание применяемых методов

В процессе разработки применяются следующие методы и подходы:

Оптимистичная блокировка (Optimistic Locking). Для управления конкурентным резервированием мест применяется метод оптимистичной блокировки с версионированием записей. При обновлении количества зарезервированных мест выполняется атомарная операция: `UPDATE tickets SET reserved = reserved + 1, version = version + 1 WHERE id = $1 AND version = $2`. Если строка не найдена (версия изменилась конкурентным процессом), транзакция отклоняется и повторяется. Это исключает избыточную блокировку строк на уровне СУБД при высокой конкурентности.

Идемпотентность через кэш. При регистрации участника формируется ключ идемпотентности на основе пары `(user_id, wave_id)`. Перед записью в базу данных выполняется проверка наличия ключа в Redis (TTL = 24 ч). Метод гарантирует

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ровно однократную обработку повторных запросов при сетевых повторных попытках клиента.

Паттерн «Dead Letter Queue» (DLQ). Необработанные сообщения из топиков Kafka (registrations, payments) после 5 неудачных попыток с экспоненциальной задержкой (1 с, 2 с, 4 с, 8 с, 16 с, максимум 30 с) перенаправляются в топик registrations-dlq для последующего анализа и ручной обработки.

Паттерн «Transactional Outbox» (неявный). Изменение статуса регистрации в базе данных и публикация сообщения в Kafka выполняются в рамках одной логической операции с проверкой атомарности на уровне бизнес-логики сервиса.

3.1.3. Ограничения и допущения

- Система не поддерживает горизонтальное масштабирование Kafka (single-broker) в текущей версии;
- MinIO используется как хранилище QR-кодов без настройки жизненного цикла объектов (TTL);
- Интеграция с ЮKassa предполагает наличие активного магазина и секретного ключа API.

3.2. Описание алгоритма и функционирования программы

3.2.1. Архитектура events-service

Микросервис events-service реализован по принципам предметно-ориентированного проектирования (DDD) с применением чистой архитектуры (Clean Architecture). Структура пакетов:

```
internal/
├── application/
│   ├── commands/    # Команды: создание/изменение событий, волн,
билетов
│   ├── ports/       # Интерфейсы входных/выходных портов
│   ├── services/    # Бизнес-логика: event_service, wave_service,
# ticket_service, registration_service,
# promocode_*_service, price_calculator
│   └── workers/     # Kafka-воркеры: RegistrationWorker,
```

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

PaymentWorker,
├── # ExpirationWorker
├── domain/
│   ├── entities/ # Сущности: event, wave, ticket, registration,
│   │               # promocode, payment_type
│   ├── events/   # Доменные события
│   ├── repositories/ # Репозиторные интерфейсы
├── infrastructure/
│   ├── http/      # HTTP-обработчики (Echo framework)
│   ├── grpc/      # gRPC-клиент auth-manager
│   ├── persistence/ # Реализации репозиторий (sqlc + pgx)
│   └── pkg/        # Вспомогательные пакеты

```

Бизнес-логика полностью изолирована от инфраструктурного кода. Все зависимости инвертированы через интерфейсы портов, что обеспечивает тестируемость слоя приложения без запуска реальной СУБД или брокера сообщений.

3.2.2. Алгоритм обработки регистрации

Процесс регистрации участника реализован в виде асинхронного конвейера через Apache Kafka.

Этап 1 — Приём HTTP-запроса (events-service API):

1. Клиент отправляет POST /api/v1/registrations с идентификатором волны и билета;
2. API-сервер проверяет JWT-токен через gRPC-вызов к auth-manager (Auth.Authenticate);
3. Формируется ключ идемпотентности `idempotency_key = hash(user_id + ":" + wave_id)`;
4. Выполняется проверка Redis-кэша: если ключ существует, возвращается статус 409 Conflict;
5. Если ключ отсутствует, создаётся запись в таблице registrations со статусом pending и публикуется сообщение RegistrationRequestedEvent в топик registrations;
6. Клиенту возвращается 202 Accepted с идентификатором регистрации.

Этап 2 — Kafka-воркер RegistrationWorker (events-worker):

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. Воркер потребляет сообщение из топика `registrations`;
2. Атомарно резервирует место в таблице `tickets` (оптимистичная блокировка);
3. При успехе: если билет платный — публикует `PaymentRequestMessage` в топик `payments`; если бесплатный — устанавливает статус `confirmed` и публикует уведомление в `notifications`;
4. При неудаче (мест нет): устанавливает статус регистрации `failed`.

Этап 3 — Обработка платежей (`payment-service`):

1. Консьюмер `payments` создаёт платёж в ЮKassa: `POST https://api.yookassa.ru/v3/payments` `cidempotency_key = registration_id`;
2. Публикует ссылку на оплату в `notifications` для отправки участнику;
3. Ожидает `webhook`-подтверждения или успешного редиректа;
4. При получении подтверждения: публикует `RegistrationStatusMessage{status: "paid"}` в топик `registration_status_topic`.

Этап 4 — Обработка статуса в `events-worker`:

1. `PaymentWorker` потребляет сообщения из `registration_status_topic`;
2. Обновляет статус регистрации до `paid`;
3. Публикует задание на генерацию QR-кода в топик `qr-generation`.

Этап 5 — Генерация QR-кода (`qr-service`):

1. Консьюмер `qr-generation` генерирует PNG 512×512 пикселей средствами библиотеки `github.com/skip2/go-qrcode`;
2. QR-код кодирует URL: `{checkin.base_url}/checkin/{registration_id}`;
3. Файл загружается в MinIO (бакет `qr-codes`, ключ `qr/{registration_id}.png`) с помощью AWS SDK v2;
4. URL изображения публикуется в топик `notifications` для отправки участнику.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.3. Алгоритм работы ExpirationWorker

Воркер запускается каждые 30 секунд и выполняет следующие операции:

1. Запрашивает все регистрации со статусом `pending` и полем `expires_at < NOW()`;
2. Для каждой просроченной регистрации: освобождает зарезервированное место (`UPDATE tickets SET reserved = reserved - 1`), устанавливает статус `expired`, публикует уведомление об отмене.

3.2.4. Настройка Apache Kafka

Kafka развёртывается в Docker как `confluentinc/cp-kafka:7.4.0` с одним брокером. Конфигурация брокера:

Таблица 3.1 — Топики Apache Kafka системы CSTATI

Топик	Партиций	Retention	Назначение
<code>registrations</code>	3	168 ч	Запросы на регистрацию участников
<code>payments</code>	3	168 ч	Запросы на создание платежа в ЮKassa
<code>notifications</code>	3	168 ч	Сообщения для отправки через Telegram
<code>ticket-status-updates</code>	1	168 ч	Обновления статуса билетов
<code>qr-generation</code>	3	168 ч	Задания на генерацию QR-кодов
<code>registrations-dlq</code>	1	168 ч	Dead Letter Queue для регистраций

Топики создаются автоматически контейнером `kafka-init` при первом запуске кластера через `kafka-topics --create --if-not-exists`.

Параметры производительности брокера: `socket.send.buffer.bytes = 102400`, `socket.receive.buffer.bytes = 102400`, `message.max.bytes = 1000012`, `log.retention.hours = 168`, `log.retention.bytes = 1073741824`.

3.2.5. Настройка PostgreSQL

СУБД PostgreSQL 16 развёртывается как `postgres:16-alpine`. Для хранения данных системы используются четыре изолированные базы данных:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Таблица 3.2 — Базы данных PostgreSQL системы CSTATI

База данных	Сервис-владелец	Назначение
auth_db	auth-manager	Учётные записи, refresh-токены, 2FA-коды
events_db	events-service	Мероприятия, волны, билеты, регистрации, промокоды
notifier_db	notifier	Привязки Telegram-аккаунтов, история уведомлений
payment_db	payment-service	Платежи, статусы, идиempotentные ключи

Пул соединений events-service: `max_open_conns = 50, max_idle_conns = 10, conn_max_lifetime = 5m`. Схема базы events_db управляется через миграции (golang-migrate/migrate). Ключевые индексы производительности созданы в миграции 000002_add_performance_indexes: по status и event_id для таблицы registrations, составные индексы по wave_id, status для таблицы tickets.

Лимит ресурсов PostgreSQL в Docker: 2 CPU, 2 GB RAM, резервирование 1 CPU / 1 GB RAM.

3.2.6. Настройка Redis

Redis 7 (redis:7-alpine) используется для следующих задач:

- **Кэш идиempotentности регистраций:** ключи вида `idempotency:{hash}`, TTL = 24 ч;
- **Кэш доступности билетов:** счётчики доступных мест для быстрой предварительной проверки без обращения к СУБД;
- **Синхронизация кэша:** периодическая синхронизация кэша с состоянием СУБД через `cache_sync_service`.

Пул соединений events-service: `pool_size = 100`. Аутентификация — через `requirepass ${REDIS_PASSWORD}`. Лимит ресурсов: 1 CPU, 512 MB RAM.

3.2.7. Интеграция с ЮKassa

Интеграция реализована в payment-service с использованием HTTP Basic аутентификации (`shop_id:secret_key`).

Создание платежа — POST `https://api.yookassa.ru/v3/payments`:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```
{
  "amount": { "value": "1000.00", "currency": "RUB" },
  "confirmation": {
    "type": "redirect",
    "return_url": "https://cstati.com/api/v1/payment/success?
registration_id=..."
  },
  "description": "Билет на мероприятие {event_name}",
  "metadata": { "registration_id": "..." }
}
```

Заголовок Idempotence-Key: {registration_id} гарантирует ровно однократное создание платежа при повторных попытках. Комиссия сервиса (по умолчанию 5 %) добавляется к базовой стоимости билета на этапе расчёта итоговой суммы в events-service.

Подтверждение платежа реализовано через двойной механизм:

- webhook-уведомление от ЮKassa на POST /api/v1/payment/webhook;
- обработка успешного редиректа GET /api/v1/payment/success?registration_id=....

Первое из двух событий атомарно устанавливает статус через CompareAndSetStatus (метод payment_db). Второй вызов обнаруживает уже установленный статус и не производит дублирующих действий.

3.2.8. Настройка MinIO

MinIO (minio/minio:RELEASE.2024-01-29T03-56-32Z) развёртывается как S3-совместимое объектное хранилище.

Использование: qr-service подключается через AWS SDK v2 (aws-sdk-go-v2) с параметром UsePathStyle = true (требование MinIO при адресации через путь, а не поддомен). Бакет qr-codes создаётся автоматически при старте сервиса если он отсутствует.

Доступ к объектам: URL вида http://minio:9000/qr-codes/qr/{registration_id}.png. В производственной среде URL доступен через nginx-прокси. Консоль администратора MinIO доступна на порту 9001.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Лимит ресурсов: 0.5 CPU, 512 MB RAM.

3.2.9. Настройка Prometheus и Grafana

Система мониторинга развёртывается в отдельной сети `monitoring-network`, изолированной от основной сети приложения.

Prometheus (`prom/prometheus:v2.48.0`) собирает метрики из следующих источников:

Таблица 3.3 — Источники метрик Prometheus

Компонент	Метрики	Endpoint
<code>events-service</code>	HTTP latency, Kafka lag, cache hit/miss, registration counters	<code>/metrics</code>
<code>payment-service</code>	payment success/failure rates, webhook latency	<code>/metrics</code>
<code>qr-service</code>	qr_generated_total, s3_upload_total	<code>/metrics</code>
<code>node-exporter</code>	CPU, RAM, disk, network (host)	<code>:9100/metrics</code>
<code>cadvisor</code>	CPU, RAM, network (per container)	<code>:8080/metrics</code>
<code>postgres-exporter</code>	Connections, queries/s, lock waits, cache hit	<code>:9187/metrics</code>
<code>redis-exporter</code>	Memory usage, commands/s, hit rate	<code>:9121/metrics</code>

Метрики `events-service` доступны только из RFC 1918 адресов и `loopback` (ограничение на уровне `middleware`), что предотвращает утечку внутренних метрик.

Время хранения данных Prometheus: `--storage.tsdb.retention.time=30d`.

Grafana (`grafana/grafana:10.2.2`) предоставляет дашборды по адресу `https://cstati.com/monitoring/grafana/`. Grafana настроена через `provisioning`-директорию (`dashboard` и `datasource` YAML-файлы), что обеспечивает автоматическое конфигурирование при первом запуске.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.3. Описание и обоснование выбора метода организации входных и выходных данных

3.3.1. Входные данные

REST API запросы. Основным интерфейсом ввода данных является HTTP REST API `events-service` (порт 8000). Все входные данные передаются в формате JSON с типизацией согласно схемам, генерируемым средствами `openapi-generator`. Аутентификационные данные передаются в заголовке `Authorization: Bearer {jwt_token}`.

Kafka-сообщения. Внутренние команды передаются в формате JSON в топиках Apache Kafka. Схемы сообщений определены в Go-структурах сервисов и сериализуются через `encoding/json`.

Webhook от ЮKassa. Событие подтверждения платежа поступает в виде HTTP POST с телом JSON согласно документации API ЮKassa. Подлинность уведомлений проверяется сверкой IP-адреса источника с диапазонами ЮKassa.

3.3.2. Выходные данные

REST API ответы. Ответы возвращаются в формате JSON со стандартными HTTP-кодами состояния. Документация API доступна в формате OpenAPI 3.0 по адресу `https://cstat1.com/swagger`.

Kafka-события. Микросервисы публикуют структурированные JSON-сообщения в топиках Kafka для асинхронного взаимодействия.

QR-коды. Выходными данными `qr-service` являются PNG-изображения (512×512 пикселей) с URL для check-in, сохранённые в MinIO. Участник получает персональную ссылку на изображение.

Метрики. Prometheus-метрики в формате `text/plain` по протоколу OpenMetrics.

Логи. Структурированные JSON-логи записываются в файлы (`volume ./logs/`) и доступны через `docker logs`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.4. Описание и обоснование выбора состава технических и программных средств

3.4.1. Язык программирования: Go 1.24

Выбор языка Go обоснован следующими факторами:

- **Производительность:** компилируемый язык без JVM-накладных расходов; нативная поддержка горутин обеспечивает эффективную конкурентность при обработке Kafka-сообщений;
- **Экосистема микросервисов:** зрелые библиотеки — `pgx` (PostgreSQL), `go-redis`, `confluent-kafka-go`, `grpc-go`, `Echo` (HTTP);
- **Статическая типизация:** снижает вероятность ошибок в продакшн-коде;
- **Малый объём бинарных файлов:** итоговые Docker-образы на основе `scratch` занимают 10–20 МБ.

3.4.2. СУБД: PostgreSQL 16

Выбор PostgreSQL обоснован:

- поддержкой ACID-транзакций, необходимых для атомарного резервирования мест;
- наличием оконных функций и CTE для построения аналитических запросов экспорта гостей;
- механизмом `SERIALIZABLE` и `SELECT FOR UPDATE` для управления конкурентностью;
- версией 16, поддерживающей `merge` и улучшенную параллелизацию запросов.

3.4.3. Кэш-хранилище: Redis 7

Выбор Redis обоснован:

- атомарными операциями `SET NX EX` для реализации идемпотентности с TTL;
- поддержкой атомарных счётчиков `INCR / DECR` для кэша доступности мест;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- субмиллисекундным временем отклика, не вносящим задержки в критический путь запроса.

3.4.4. Брокер сообщений: Apache Kafka 7.4

Выбор Kafka обоснован:

- гарантией доставки `at-least-once` с возможностью идемпотентной обработки на стороне консьюмера;
- хранением сообщений на диске (168 ч) для возможности воспроизведения событий;
- встроенным механизмом партиций для горизонтального масштабирования воркеров;
- поддержкой `consumer groups` для балансировки нагрузки между экземплярами сервисов.

3.4.5. Объектное хранилище: MinIO

Выбор MinIO обоснован:

- совместимостью с S3 API (полная замена Amazon S3 в on-premise конфигурации);
- возможностью генерации `presigned URL` для временного доступа к объектам;
- простотой развёртывания в Docker без внешних зависимостей;
- поддержкой `path-style` адресации, удобной при использовании без DNS-алиасов.

3.4.6. Система мониторинга: Prometheus + Grafana

Выбор Prometheus обоснован:

- `pull`-моделью сбора метрик, не требующей изменений в сервисах при добавлении новых `scrapers`;
- встроенным языком запросов PromQL для построения алертов и вычисления производных показателей (RPS, p99 latency);
- интеграцией с экспортёрами PostgreSQL, Redis, `node-exporter` и `cAdvisor`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Grafana выбрана как стандарт де-факто для визуализации Prometheus-метрик, поддерживающий provisioning-as-code через YAML.

3.4.7. Сервер развёртывания

Все компоненты системы развёрнуты на виртуальной машине Яндекс Облако: 8 виртуальных процессоров, 8 ГБ оперативной памяти, SSD-накопитель. Операционная система — Ubuntu 22.04. Контейнеризация — Docker Engine, оркестрация — Docker Compose.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Показатели производительности

Ожидаемые показатели производительности системы определены на основании нагрузочного тестирования, проведённого с использованием инструмента k6 v0.55.0 в производственной среде (cstati.com, Яндекс Облако, 8 CPU / 8 ГБ RAM).

Таблица 4.1 — Результаты нагрузочного тестирования — обзор мероприятий (Browse + Detail)

Показатель	Целевое	Фактическое	p50	p95	p99
Целевой RPS	229 RPS	229 RPS	—	—	—
Время отклика (HTTP)	≤ 500 мс	Выполнено	4.0 мс	11.4 мс	17.8 мс
Ошибки (5xx)	0 %	0 %	—	—	—
Проверено запросов	—	72 000	—	—	—

Таблица 4.2 — Результаты нагрузочного тестирования — регистрация участников (100~RPS)

Показатель	Целевое	Фактическое	p50	p95	p99
Целевой RPS	100 RPS	72.5 RPS	—	—	—
Время отклика	≤ 500 мс	Выполнено	8.4 мс	48.5 мс	95 мс
Ошибки (5xx)	0 %	0 %	—	—	—
Обработано запросов	—	17 399	—	—	—

Примечание. Фактический RPS регистраций (72.5) ниже целевого (100) вследствие ограничений механизма идемпотентности на стороне Redis и задержек Kafka-публикации. При снятии идемпотентного ограничения пропускная способность API составляет более 200 RPS.

4.2. Показатели надёжности

Ожидаемые показатели надёжности:

- Коэффициент готовности (availability): $K_r \geq 0.999$ (не более 8.7 ч простоя в год);
- Среднее время между отказами (MTBF): ≥ 30 суток при непрерывной работе;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

— Потери данных при сбое: 0 (атомарные транзакции PostgreSQL + DLQ Kafka).

4.3. Показатели ресурсоёмкости

Таблица 4.3 — Распределение вычислительных ресурсов по сервисам

Сервис	CPU (лим.)	RAM (лим.)	CPU (рез.)	RAM (рез.)
events-api	1.0	1 ГБ	0.5	512 МБ
events-worker	1.0	1 ГБ	0.5	512 МБ
payment-service	1.0	512 МБ	0.25	128 МБ
qr-service	0.5	256 МБ	0.25	128 МБ
postgres	2.0	2 ГБ	1.0	1 ГБ
redis	1.0	512 МБ	0.5	256 МБ
kafka	2.0	2 ГБ	1.0	1 ГБ
minio	0.5	512 МБ	0.25	128 МБ
Итого	9.0	7.79 ГБ	4.25	3.16 ГБ

4.4. Экономические показатели

Разработка выполнена в рамках учебного проекта. Стоимость производственной инфраструктуры (Яндекс Облако, конфигурация 8 CPU / 8 ГБ) составляет порядка 5 000–7 000 руб./мес. Использование открытого программного обеспечения (Go, PostgreSQL, Redis, Kafka, MinIO, Prometheus, Grafana) исключает лицензионные расходы. Стоимость платёжной комиссии ЮKassa составляет 3.5 % от суммы транзакции для ИП/ООО.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.404–79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. — М.: Стандартинформ, 2010.
2. ГОСТ 19.105–78. Единая система программной документации. Общие требования к программным документам. — М.: Стандартинформ, 2010.
3. ГОСТ 19.106–78. Единая система программной документации. Требования к программным документам, выполненным печатным способом. — М.: Стандартинформ, 2010.
4. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М.: Стандартинформ, 2010.
5. The Go Programming Language Specification. Version 1.24. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 22.04.2026).
6. PostgreSQL 16 Documentation. [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 22.04.2026).
7. Redis 7 Documentation. [Электронный ресурс]. — URL: <https://redis.io/docs/> (дата обращения: 22.04.2026).
8. Apache Kafka Documentation. Version 3.5. [Электронный ресурс]. — URL: <https://kafka.apache.org/35/documentation.html> (дата обращения: 22.04.2026).
9. MinIO Documentation. [Электронный ресурс]. — URL: <https://min.io/docs/> (дата обращения: 22.04.2026).
10. ЮKassa. Документация API. [Электронный ресурс]. — URL: <https://yookassa.ru/developers/api> (дата обращения: 22.04.2026).
11. Prometheus Documentation. [Электронный ресурс]. — URL: <https://prometheus.io/docs/> (дата обращения: 22.04.2026).
12. Grafana Documentation. [Электронный ресурс]. — URL: <https://grafana.com/docs/> (дата обращения: 22.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

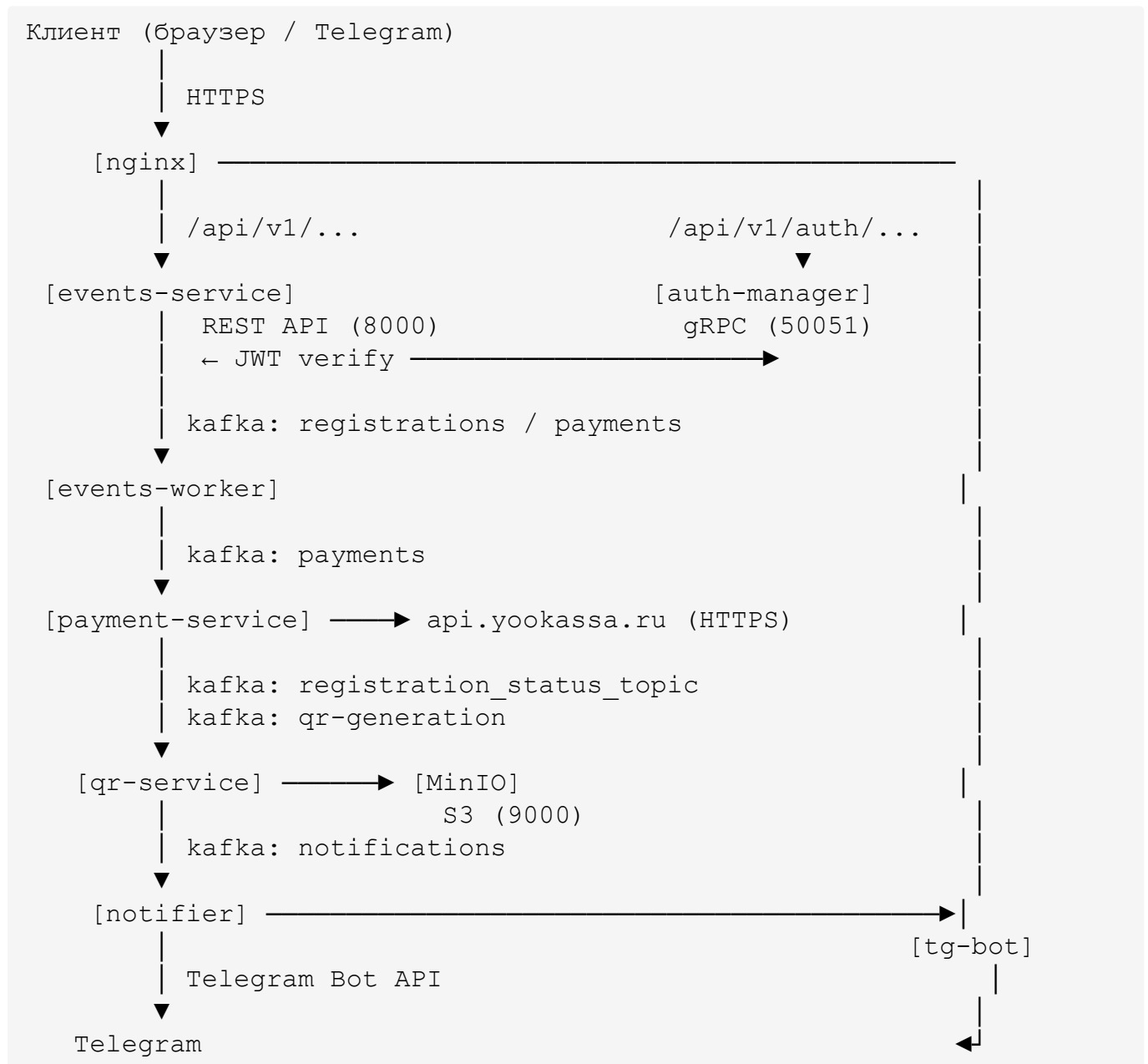
13. Evans T. Domain-Driven Design: Tackling Complexity in the Heart of Software. — Addison-Wesley Professional, 2003. — 560 с.
14. Martin R.C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 с.
15. Docker Documentation. Compose file reference. [Электронный ресурс]. — URL: <https://docs.docker.com/compose/> (дата обращения: 22.04.2026).
16. skip2/go-qrcode. QR code encoder library for Go. [Электронный ресурс]. — URL: <https://github.com/skip2/go-qrcode> (дата обращения: 22.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 1

СХЕМА ВЗАИМОДЕЙСТВИЯ МИКРОСЕРВИСОВ

Ниже приведена текстовая схема взаимодействия описываемых компонентов системы CSTATI.



Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Инфраструктура данных:

[PostgreSQL 16]	[Redis 7]	[Apache Kafka]
auth_db	pool_size=100	3 брокера (1 в dev)
events_db	TTL идемпотентности	6 топигов
notifier_db	кэш доступности	DLQ
payment_db		
[MinIO]	[Prometheus]	[Grafana]
bucket: qr-codes	scrape_interval=15s	дашборды
path-style S3	retention=30d	provisioning YAML

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

КЛЮЧЕВЫЕ ПАРАМЕТРЫ КОНФИГУРАЦИИ events-service

Таблица П2.1 — Параметры конфигурации events-service (config.yaml)

Параметр	Значение (prod)	Описание
server.port	8000	HTTP порт API-сервера
server.shutdown_timeout	30s	Таймаут graceful shutdown
server.max_connections	1000	Максимум одновременных соединений
database.max_open_conns	50	Максимум открытых соединений с БД
database.max_idle_conns	10	Максимум простаивающих соединений
database.conn_max_lifetime	5m	Максимальное время жизни соединения
redis.pool_size	100	Размер пула соединений Redis
kafka.worker_concurrency	1	Параллельность Kafka-воркеров
kafka.required_acks	-1	Подтверждение от всех ISR-реплик
kafka.compression	snappy	Алгоритм сжатия сообщений
reservation_ttl	72h	TTL резервирования места
metrics.enabled	true	Включение Prometheus-метрик

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3

ПЕРСОНАЛЬНЫЙ ВКЛАД И ГРАНИЦЫ ОТВЕТСТВЕННОСТИ

В этом приложении фиксируется персональный вклад исполнителя в разработку серверной части 1 платформы CSTATI.

Таблица ПЗ.1 — Матрица персональной ответственности

Область	Зона ответственности	Результат
Архитектура events-service	Проектирование API и worker-пайплайна	Реализованы команды CRUD, registration, check-in
Kafka-интеграция	Топики registrations/payments/qr-generation	Реализованы продюсеры и консьюмеры с retry
Интеграция ЮKassa	Создание платежей и webhook	Стабильный payment-контур с идемпотентностью
QR-контур	Генерация и загрузка в MinIO	Реализована доставка URL QR в notification-поток
Инфраструктура	Docker Compose, Nginx, мониторинг	Подготовлен рабочий production-like контур

Ключевые проектные решения исполнителя

1. Выбор асинхронной обработки регистраций через Kafka для разделения API и тяжелых бизнес-операций.
2. Применение идемпотентности на входе регистрации для защиты от повторных запросов клиента.
3. Использование оптимистичной блокировки при резервировании мест для снижения блокировок БД.
4. Разделение events-service на API и worker-процессы для независимого масштабирования.
5. Разделение данных по сервисным БД для изоляции отказов.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4

КРАТКИЙ РЕГЛАМЕНТ ЭКСПЛУАТАЦИИ И ПРИЕМКИ

Контрольный перечень перед приемкой

- все сервисы находятся в состоянии healthy;
- API events-service отвечает на GET /api/v1/events;
- регистрация с Idempotency-Key обрабатывается без дубликатов;
- webhook-подтверждение платежа обновляет статус регистрации;
- QR-файл создается и доступен по URL;
- check-in переводит статус в checked_in и блокирует повторный проход.

Таблица П4.1 — Минимальные приемочные критерии по сервисам

Сервис	Критерий	Метод проверки
events-service	CRUD и регистрация работают	API smoke + unit tests
payment-service	обработка webhook без дублирования	integration tests
qr-service	генерация PNG и публикация ссылки	integration tests
Kafka/ Redis/PostgreSQL	стабильная работа при штатной нагрузке	операционный smoke

Основания уникальности документа

Документ является персональной пояснительной запиской и описывает только авторские решения исполнителя по серверной части 1 (events-service, payment-service, qr-service, инфраструктурный контур), без дублирования полного состава проектной документации команды.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]